Data Visualization in Python:

Yu-Che Tsai 蔡育哲 統計109

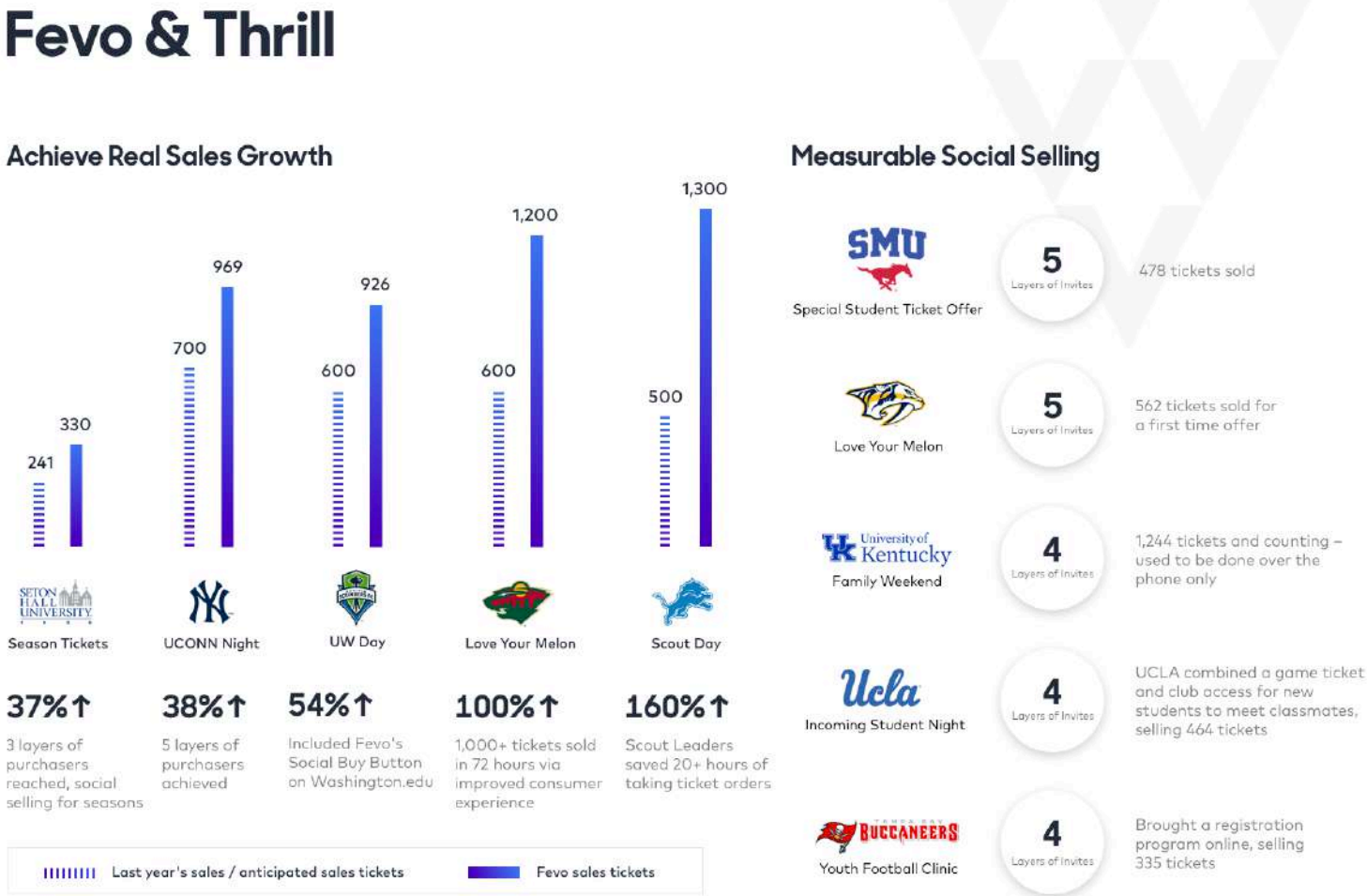
National Cheng Kung University, Taiwan

INTRODUCTION TO COMPUTERS

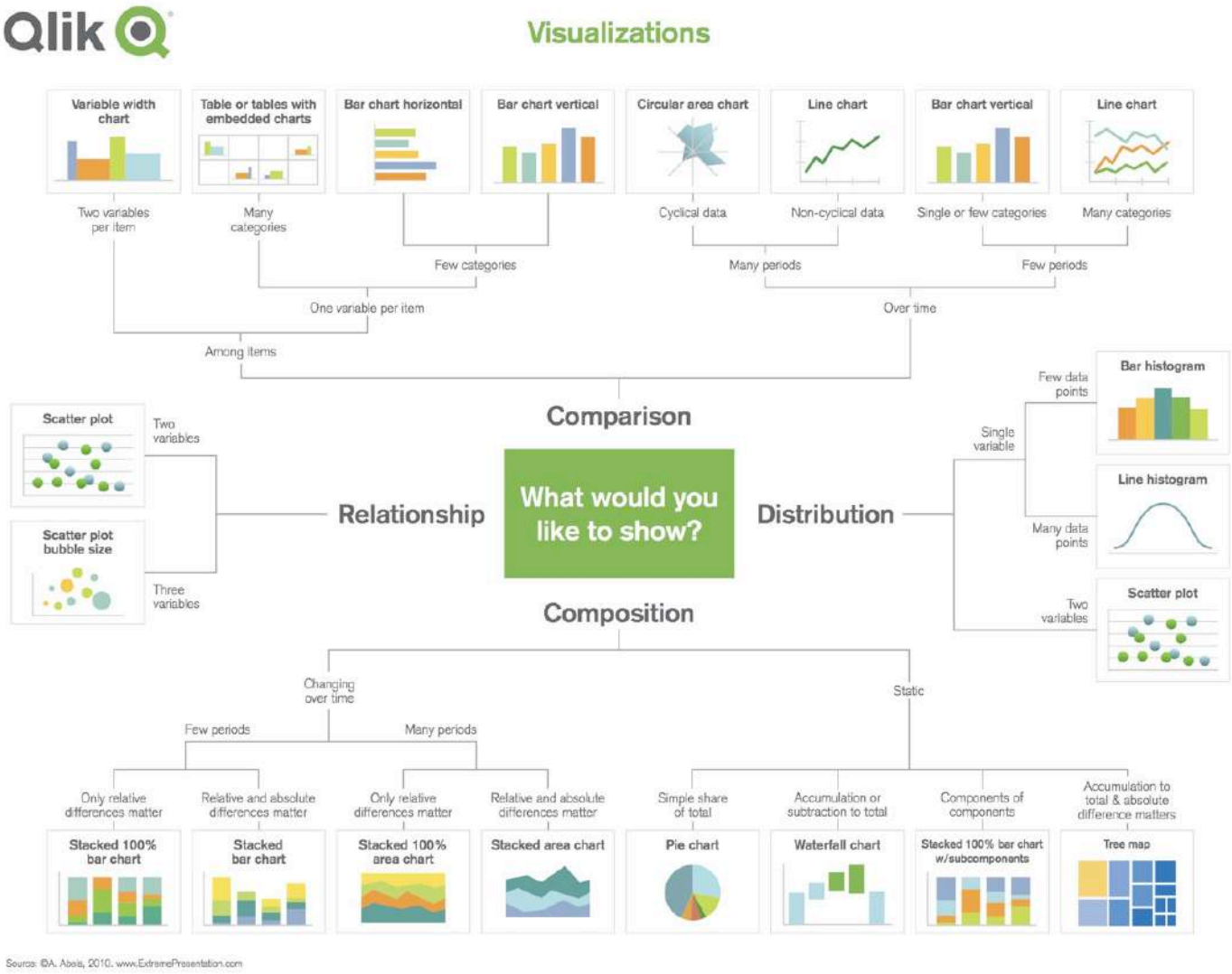
Why data visualization is important?

- The era of Big data has come, we create billions of data in our daily life such as trading records, click logs
- When you have millions or billions of data, it is essential to make plots to explore insights from these dataset
- Studies shows that...
  - The speed of human brain to process figures is 60000 times faster than process text data
  - 90% of the information were delivered to our brain visually
  - people could absorb 80% of figures but only 20% of text data
- Website including beautiful figures gains 94% click through rate more than those only adopt texts

Why data visualization is important?



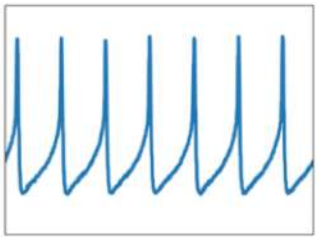
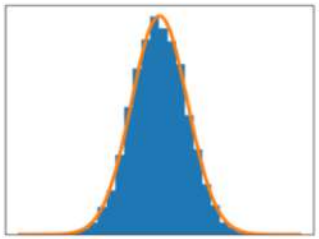
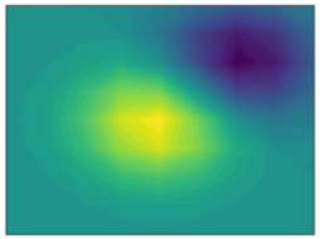
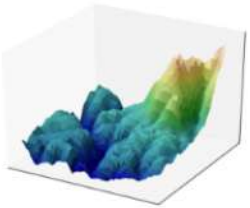
Why data visualization is important?



A useful package for data visualization

- Matplotlib is one of the best Python 2D plotting library which has been used worldwide for decades

matplotlib

What are modules and packages?

- Module is a file that contains some utilities such as Classes, Functions and Variables
  - Actually, each .py file could be a modules
  - You can import the utilities from another file (.py file)
- Packages are composed of multiple modules
  - We aggregate modules with similar purpose or topic together
  - All the modules were collected in the same folder
- With modules & packages, you don't need to define or write all the functions and variables in one file
  - Simplify your code and make it easier to understood
  - convenient for people to collaborate

What are modules and packages?

main.py

```
1 from myutils import add,print_func
2
3 x = 5
4 y = 6
5 print(add(x,y))
6 print_func(add(x,y))
```

import functions from myutils.py

myutils.py

```
1 def add(x,y):
2     result = x+y
3     return result
4
5 def print_func(string):
6     print("The word you input is %s"%string)
```

What are modules and packages?

main.py

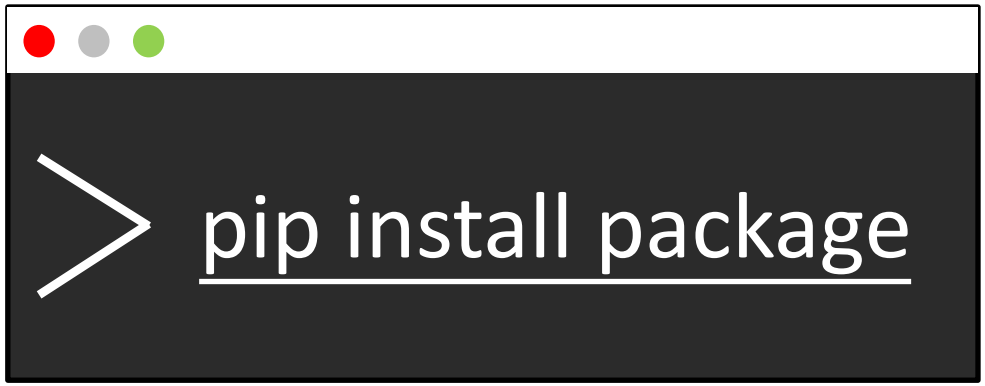
```
1 from myutils import pi,print_func,calculate_circle
2
3 r = 7
4 area = calculate_circle(pi,r)
5 print_func(area)
```

myutils.py

```
1  pi = 3.1415926
2  def add(x,y):
3      result = x+y
4      return result
5
6  def print_func(string):
7      print("The word you input is %s"%string)
8
9  def calculate_circle(pi,r):
10     return pi*r**2
```

How to use/install others packages?

- We can import or use others packages by installing the packages via pip
  - Use pip install xxx on your CMD or terminal
  - Check installed package with pip list
- Numpy: A useful tools for math and statistics calculation
- Pandas: Reading files and process data like excel



How to use/install others packages?

```
$ pip list
Package    Version
---------- -------
pip        19.3.1
setuptools 42.0.2
wheel      0.33.6
```

First, type pip list to check your installed packages

```
$ pip install numpy
Collecting numpy
  Downloading https://files.pythonhosted.org/packages/22/99/36e3408ae2cb8b72260de4e538196d17736
d7fb82a1086cb2c21ee156ddc/numpy-1.17.4-cp36-cp36m-macosx_10_9_x86_64.whl (15.1MB)
    |                                | 15.1MB 2.2MB/s
Installing collected packages: numpy
Successfully installed numpy-1.17.4
```

```
$ pip list
Package    Version
---------- -------
numpy      1.17.4
pip        19.3.1
setuptools 42.0.2
wheel      0.33.6
```

After successfully installed, type again to make sure it installed

What you need today

- Make sure you install matplotlib and numpy
  - pip install numpy matplotlib
  - type pip list
- Import these packages
  - Usually, we shorten the name of the packages
  - np for numpy
  - plt for matplotlib

```
$ pip list
Package         Version
--------------- -------
cycler          0.10.0
kiwisolver      1.1.0
matplotlib      3.1.2
numpy           1.17.4
```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Quick introduction for Numpy

- NumPy is the fundamental package for scientific computing with Python
- It helps us to make element-wise calculation
- Matrix computation and statistical method are provided

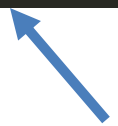
```
1 import numpy as np
2
3 x = np.array([1,2,3,4])
4 x_plus = x + 1 # np.array([2,3,4,5])
5 x_mul = x * 2 # np.array([2,4,6,8])
6 x_power = x**2 # np.array([1,4,9,16])
7
8 y = np.array([5,3,7,6])
9 print(x+y) # np.array([6,5,10,10])
```

Overview of plotting in python

- Step 1: Import matplotlib package
- Step 2: Type your plot functions
  - Line plot, Bar plot, Scatter plot
- Step 3: Add more information on the figure
  - Title, label for x/y axis,
- Step 4: plt.show()
  - Don’t forget to add this line!!

Overview of plotting in python

```
1  import matplotlib.pyplot as plt
2
3
4      Make your plot here!
5
6
7  plt.show()
```

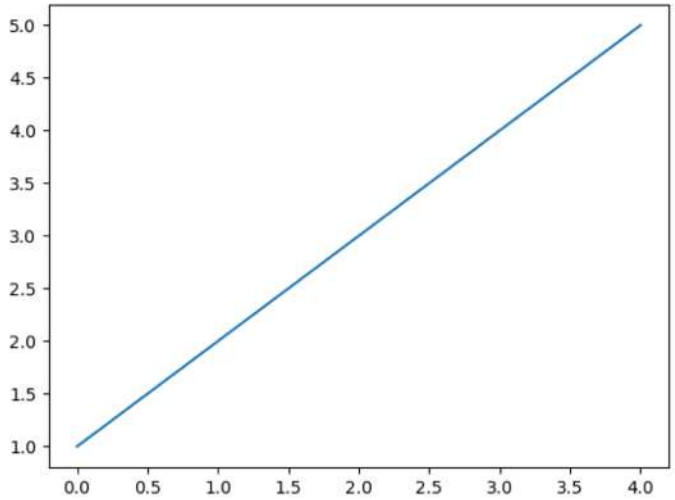


Make sure your plots shown on the screen!

Line plot

- To draw a line plot, use plt.plot(x,y)
  - if only one input is assigned, it would be plot on the y-axis

```
1  import matplotlib.pyplot as plt
2  y = [1,2,3,4,5]
3  plt.plot(y)
4  plt.show()
```

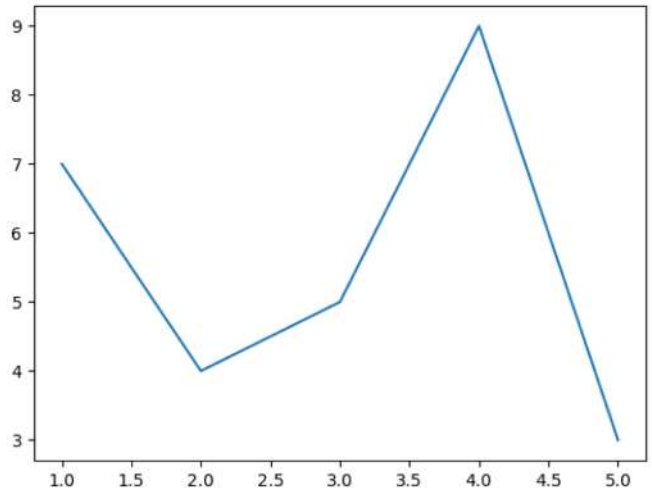


Line plot

- To draw a line plot, use plt.plot(x,y)
  - if only one input is assigned, it would be plot on the y-axis

```
1 import matplotlib.pyplot as plt
2 x = [1,2,3,4,5]
3 y = [7,4,5,9,3]
4 plt.plot(x,y)
5 plt.show()
```

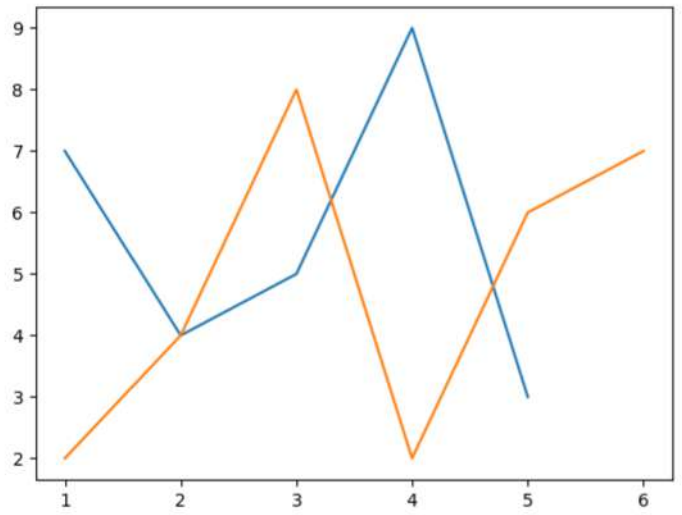
***len(x)==len(y)**



Line plot

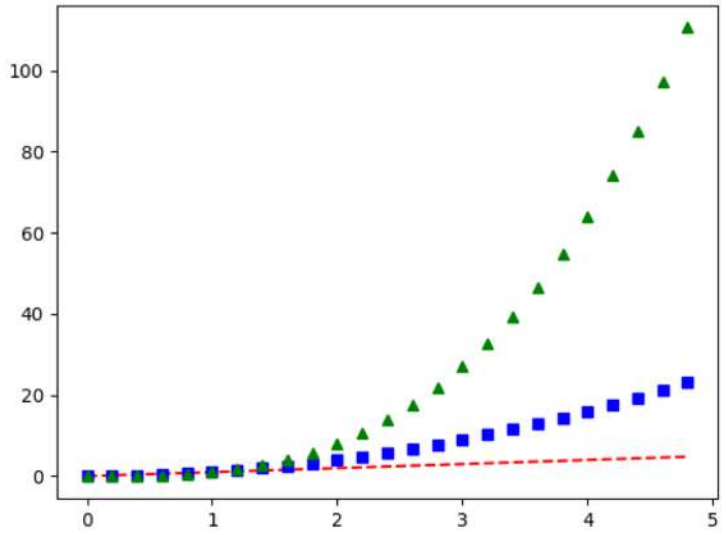
- To draw multiple line plot, use plot functions several times before show function
  - Different line would be assigned different colors

```
1 import matplotlib.pyplot as plt
2 x = [1,2,3,4,5]
3 y = [7,4,5,9,3]
4 plt.plot(x,y)
5 x2 = [1,2,3,4,5,6]
6 y2 = [2,4,8,2,6,7]
7 plt.plot(x2,y2)
8 plt.show()
```



Line Properties

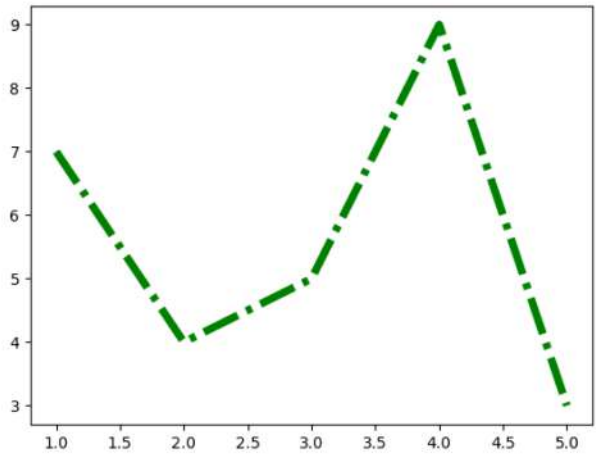
- Linewidth 寬度
  - linewidth(lw) = 2.0 / 3.5 / 6.0
- Linestyle 線條樣式
  - linestyle(ls) = “-” / “- -” / “-.”
- Color 顏色
  - color(c) = “red” / “yellow”
- Marker 點的樣式
  - marker = “s” / ”^” / ”*”



Line Properties

- You can add additional properties after typing the plot function as parameters

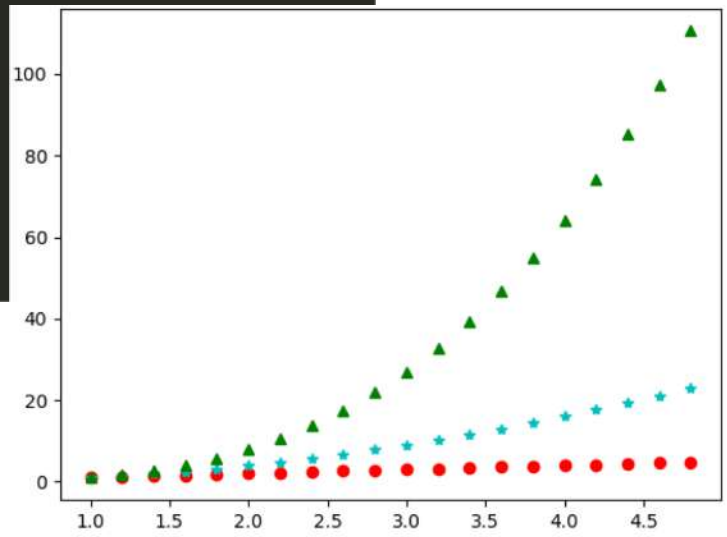
```
1 import matplotlib.pyplot as plt
2 x = [1,2,3,4,5]
3 y = [7,4,5,9,3]
4 plt.plot(x,y,lw=5.0,ls="-.",color="g")
5 plt.show()
```



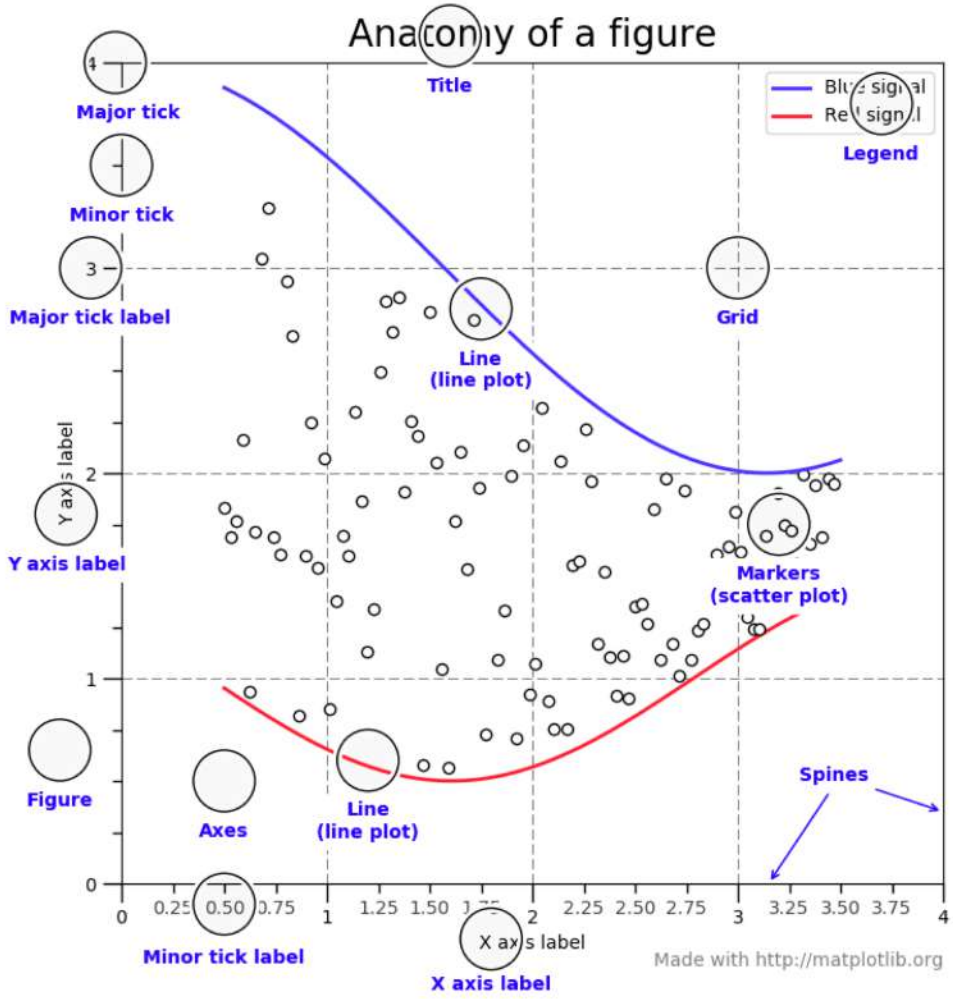
Line Properties

- You can add additional properties after typing the plot function as parameters

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  y = np.arange(1,5,0.2)
4  plt.plot(y,"ro")
5  plt.plot(y**2,"c*")
6  plt.plot(y**3,"g^")
7  plt.show()
```



Add information on the figure

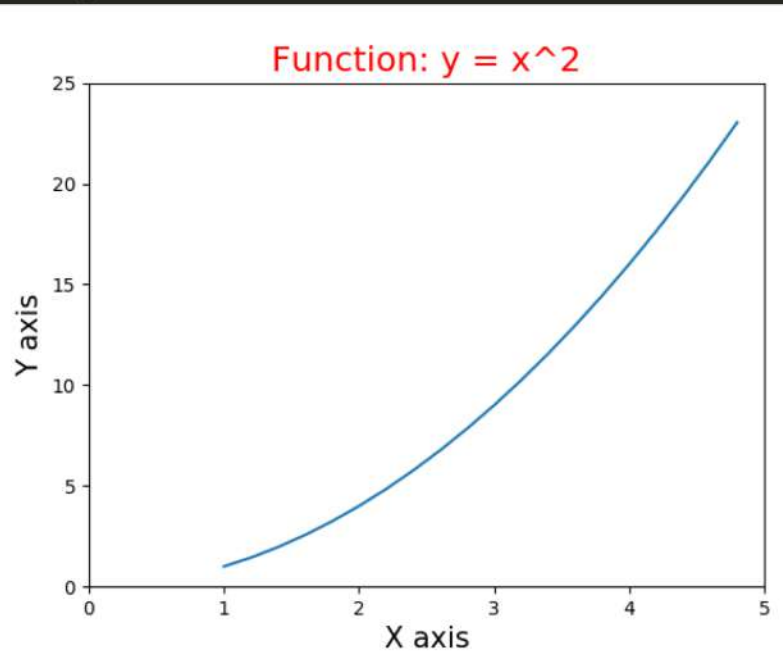


Add information on the figure

- 圖片長寬
  - plt.figure(figsize=(width, length))
- 圖片標題
  - plt.title(“This is the title of the figure”)
- X軸標題
  - plt.xlabel(“X axis”)
- X軸範圍
  - plt.xlim(lower bond, upper bond)
- 顯示圖例
  - plt.legend()

Add information on the figure

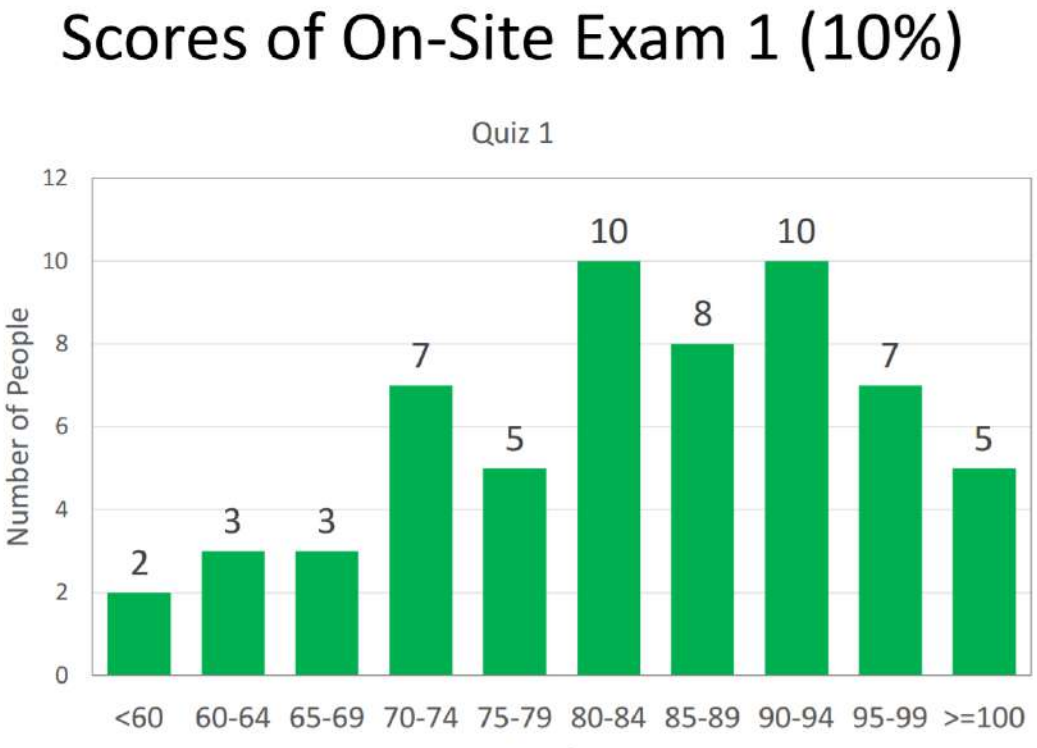
```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  x = np.arange(1,5,0.2)
4  plt.plot(x,x**2)
5  plt.title("Function: y = x^2",fontsize = 18,color = "r")
6  plt.xlabel("X axis",fontsize=15)
7  plt.ylabel("Y axis",fontsize=15)
8  plt.xlim(0,5)
9  plt.ylim(0,25)
10 plt.show()
```

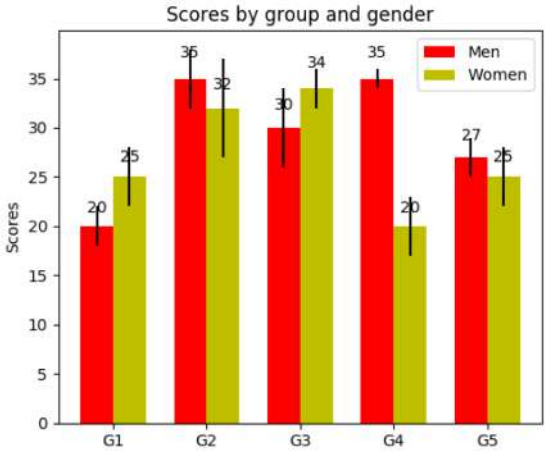


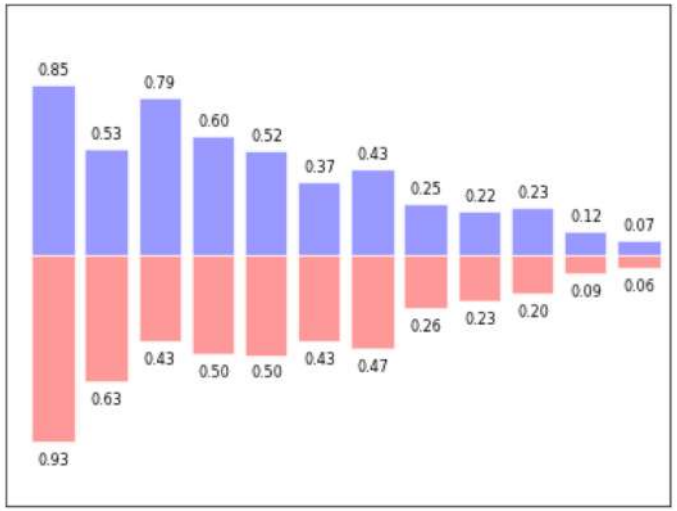
Many kinds of plots

- All the parameters could be applied in other kinds of plot in matplotlib
  - Bar plot
  - Scatter plot
  - Pie chart
  - Histogram
- The most easiest way to make a plot
  - plt.xxxplot(data)

Bar plot


Scores of On-Site Exam 1 (10%)
Quiz 1
Number of People
0
2
4
6
8
10
12
2
3
3
7
5
10
8
10
7
5
<60
60-64
65-69
70-74
75-79
80-84
85-89
90-94
95-99
>=100

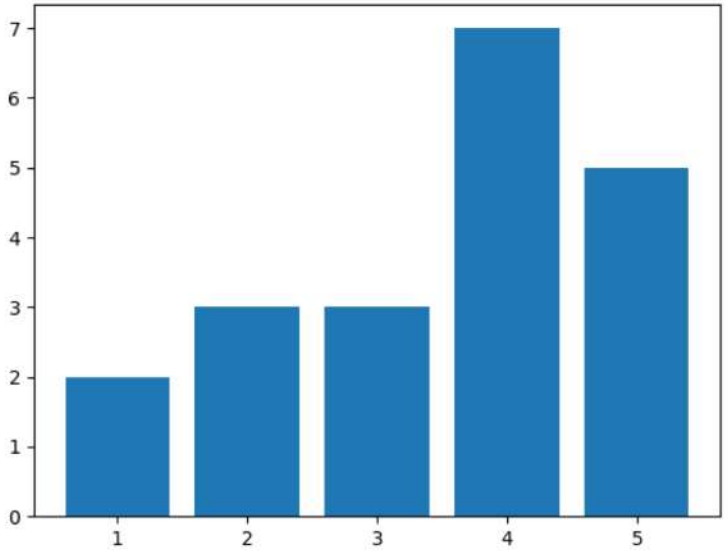

Scores by group and gender
Men
Women
Scores
0
5
10
15
20
25
30
35
20
25
30
32
30
34
35
20
27
25
G1
G2
G3
G4
G5


0.85
0.53
0.79
0.60
0.52
0.37
0.43
0.25
0.22
0.23
0.12
0.07
0.93
0.63
0.43
0.50
0.50
0.43
0.47
0.26
0.23
0.20
0.09
0.06

Bar plot

- Given the X, Y pair
  - use plt.bar(x,y) to make a bar plot

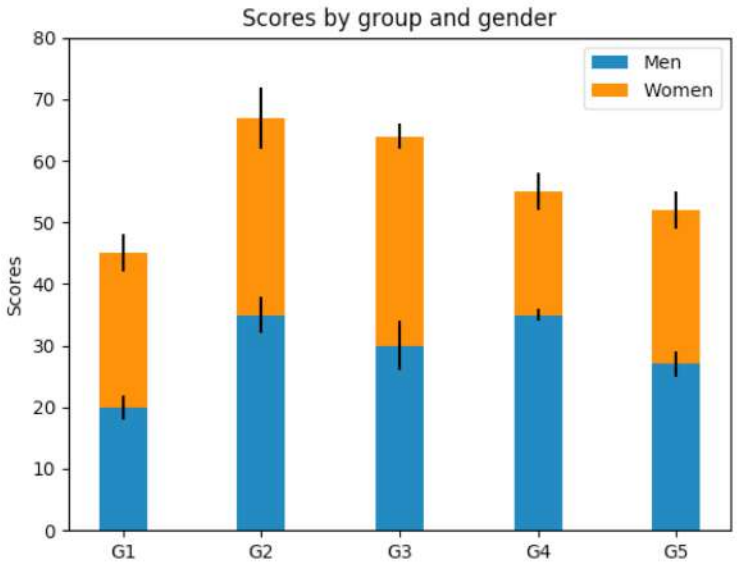
```
1 import matplotlib.pyplot as plt
2 x=[1,2,3,4,5]
3 y = [2,3,3,7,5]
4 plt.bar(x,y)
5 plt.show()
```



Bar plot

- Stacking bars with multiple bar() functions
- Add error ticks on y-axis

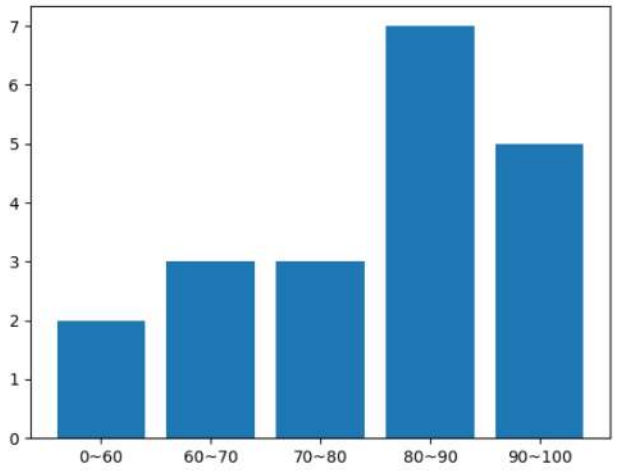
```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4
5  N = 5
6  menMeans = (20, 35, 30, 35, 27)
7  womenMeans = (25, 32, 34, 20, 25)
8  menStd = (2, 3, 4, 1, 2)
9  womenStd = (3, 5, 2, 3, 3)
10 ind = np.arange(N)
11 width = 0.35
12
13 p1 = plt.bar(ind, menMeans, width, yerr=menStd)
14 p2 = plt.bar(ind, womenMeans, width,
15             bottom=menMeans, yerr=womenStd)
16
17 plt.ylabel('Scores')
18 plt.title('Scores by group and gender')
19 plt.xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
20 plt.yticks(np.arange(0, 81, 10))
21 plt.legend(['Men', 'Women'])
22 plt.show()
```



Change Xticks in Bar plot

- X-axis might have category meanings instead of numbers
  - Use plt.xticks(x,label_name) to change xticks

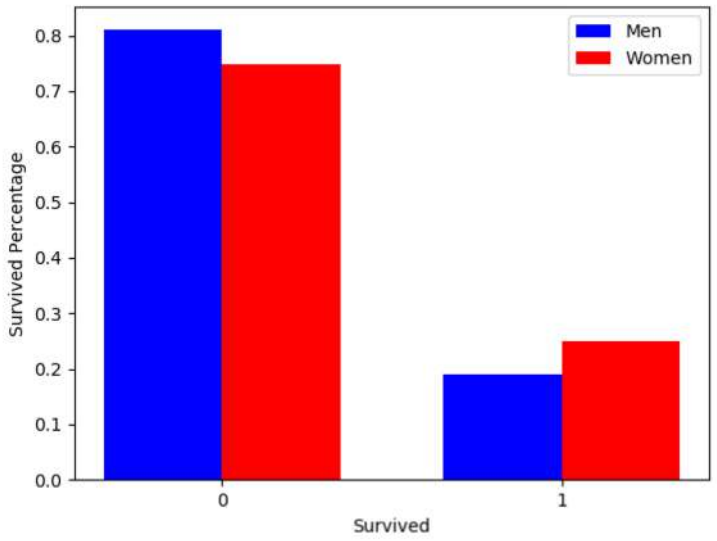
```
1 import matplotlib.pyplot as plt
2 x=[1,2,3,4,5]
3 score = ["0~60","60~70","70~80","80~90","90~100"]
4 number = [2,3,3,7,5]
5 plt.bar(x,number)
6 plt.xticks(x,score)
7 plt.show()
```



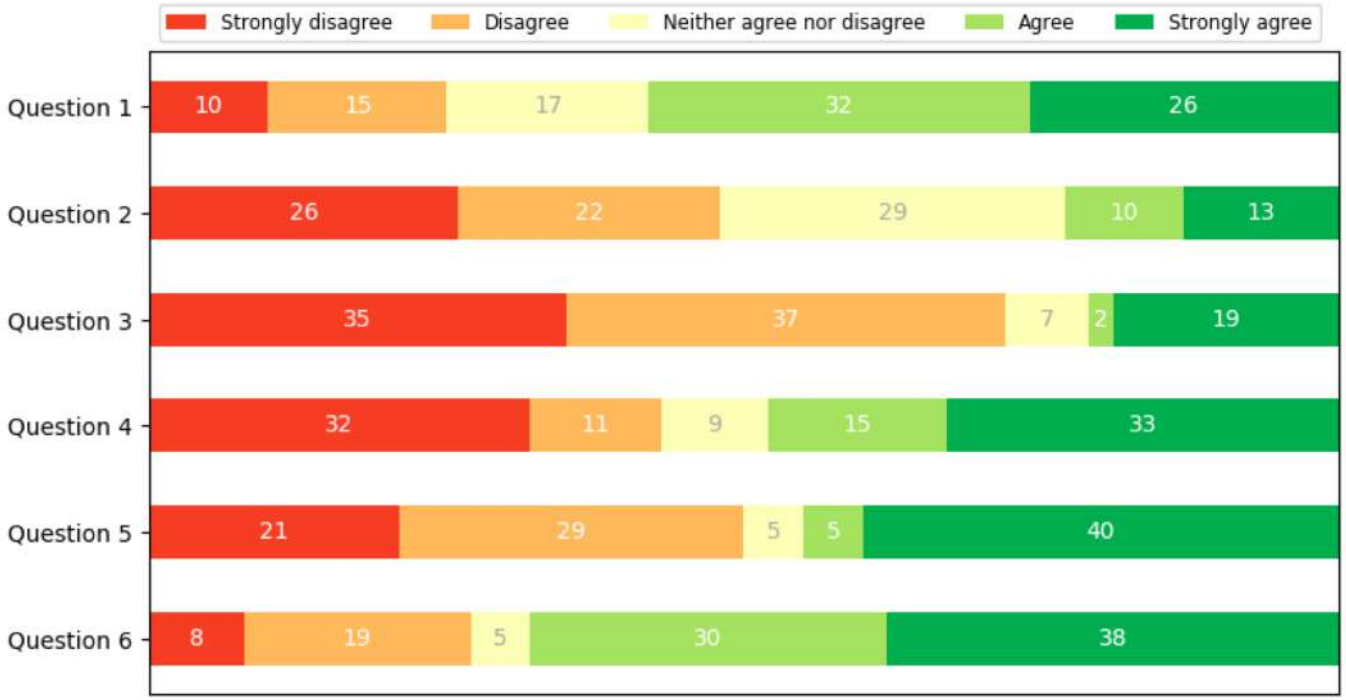
Change Xticks in Bar plot

- X-axis might have category meanings instead of numbers
  - Use plt.xticks(x,label_name) to change xticks

```
x_len = 2
x = np.arange(x_len) #x has two columns
#draw the ratio of people survived
plt.bar(x, men_survived, width=0.35, color = "b")
plt.bar(x+0.35, women_survived, width=0.35, color= "r")
#add some explanation
plt.xlabel("Survived")
plt.ylabel('Survived Percentage')
plt.xticks(x+0.35/2,[0,1])
plt.show()
```

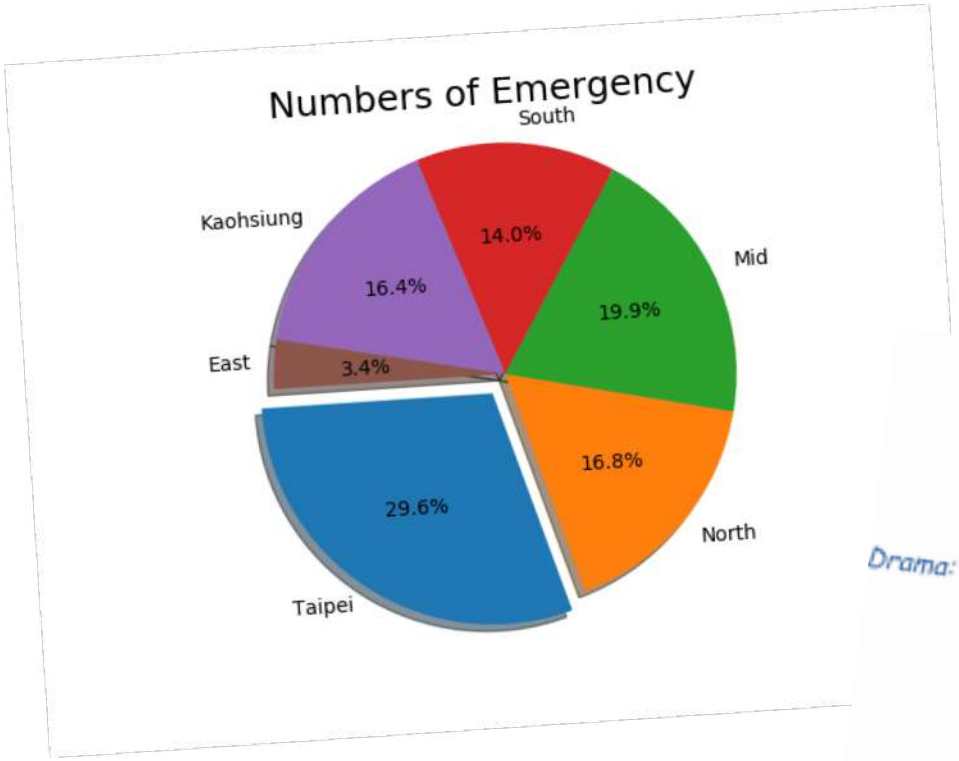


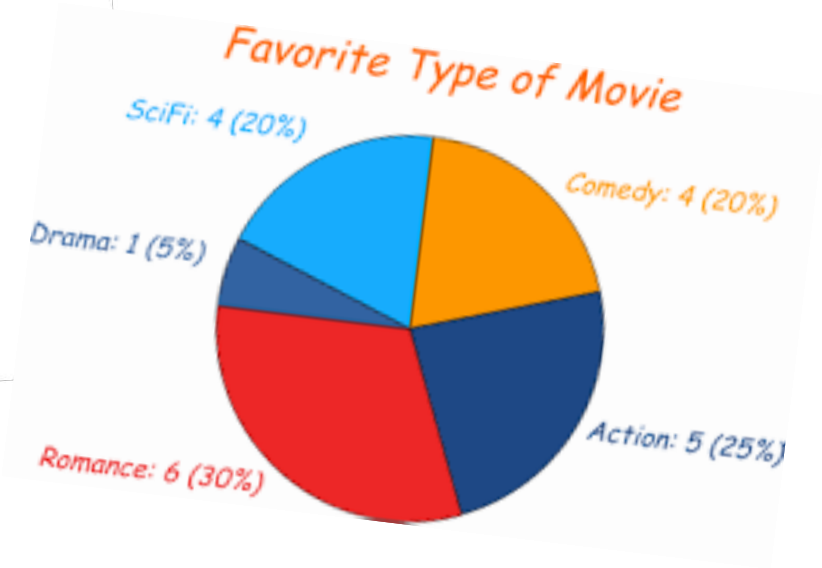
Bar plot



https://matplotlib.org/3.1.1/gallery/lines_bars_and_markers/horizontal_barchart_distribution.html#sphx-glr-gallery-lines-bars-and-markers-horizontal-barchart-distribution-py

Pie Chart

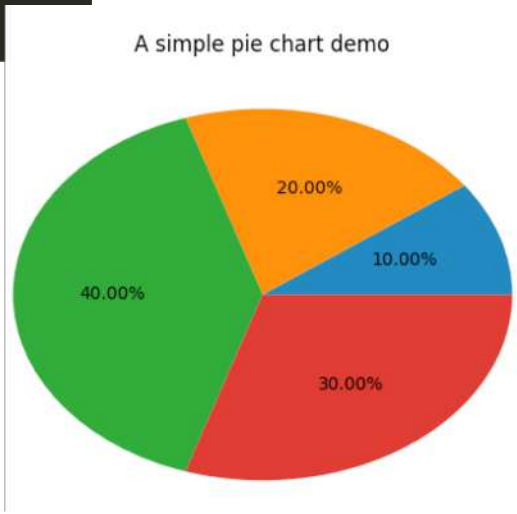

Numbers of Emergency
South
Kaohsiung
Mid
East
North
Taipei
14.0%
16.4%
19.9%
3.4%
16.8%
29.6%


Favorite Type of Movie
SciFi: 4 (20%)
Comedy: 4 (20%)
Drama: 1 (5%)
Action: 5 (25%)
Romance: 6 (30%)

Pie Chart

- It is very easy to make a pie chart in python
  - Use plt.pie(data) put all the data in a list
  - Done!

```
1  import matplotlib.pyplot as plt
2
3  size = [5,10,20,15]
4  plt.pie(size,autopct="%.2f%%")
5  plt.title("A simple pie chart demo")
6  plt.show()
```



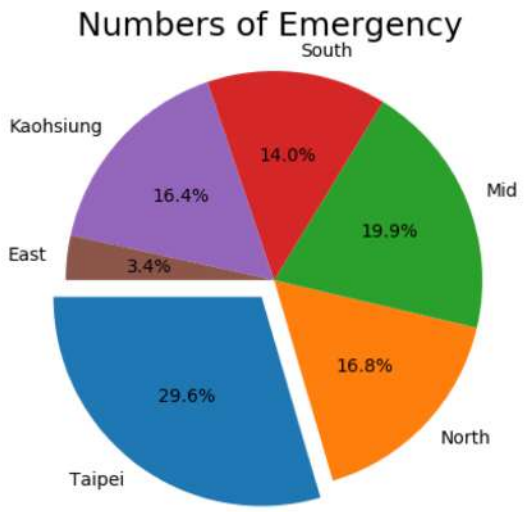
Some useful parameters for pie chart

- 圖例說明
  - labels = [“John”, “Betty”, “Marry”]
- 圓餅顏色
  - colors = [“green”, ”red”, ”yellow”]
- 凸出效果
  - explodes = [0,0,0.1,0]
- 項目百分比格式
  - autopct = “%.2f%%”
- 起始角度
  - startangle = 180

Pie Chart

- Add parameters in pie function to make chart informative

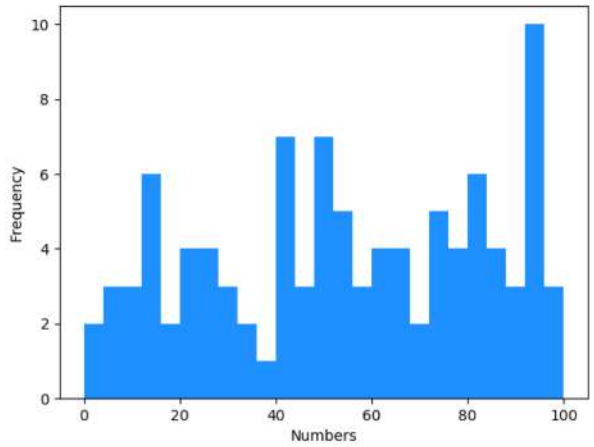
```
33 lab = "Taipei","North","Mid","South","Kaohsiung","East"
34 exp = 0.1,0,0,0,0,0
35 plt.pie(size,labels=lab,autopct = "%3.1f%%",explode =exp, startangle=180)
36 plt.axis("equal")
37 plt.title("Numbers of Emergency",fontsize=18)
38 plt.show()
```



Histogram

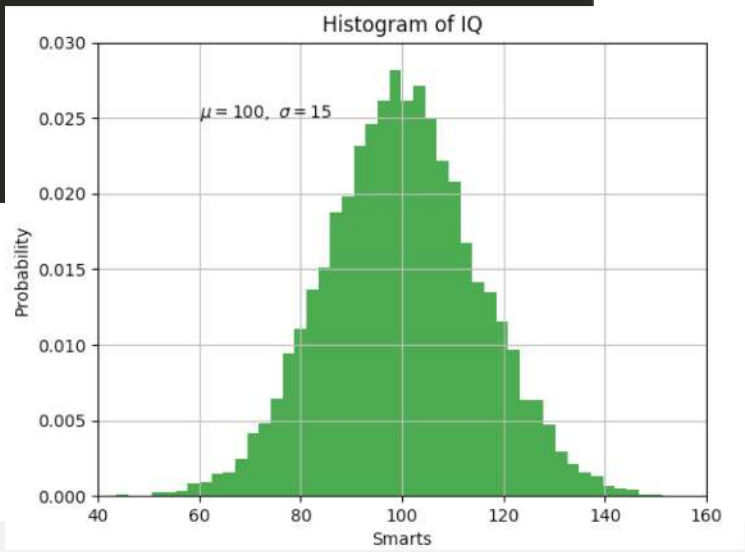
- Plot the distribution of the dataset with histogram
  - Use plt.hist()
  - Check normal distribution or left / right skew

```
1 import matplotlib.pyplot as plt
2 import random as rd
3 num = [rd.randint(0,100) for x in range(100)]
4 plt.hist(num,bins=25,color= 'dodgerblue')
5 plt.xlabel("Numbers")
6 plt.ylabel("Frequency")
7 plt.show()
```



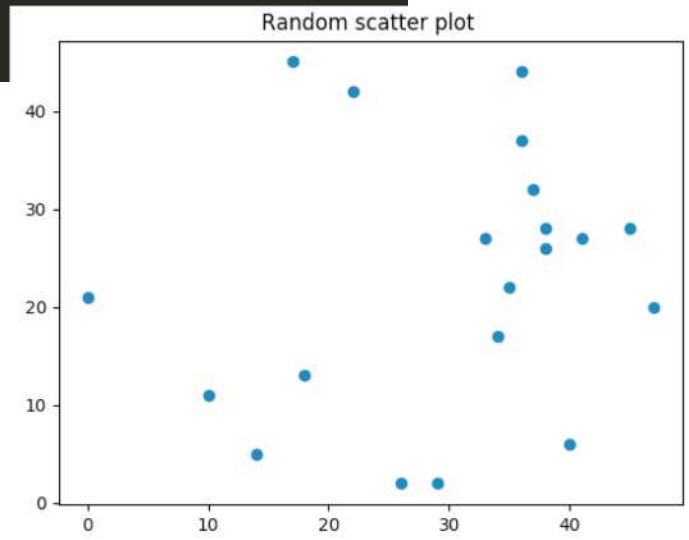
Histogram

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  mu, sigma = 100, 15
5  x = mu + sigma * np.random.randn(10000)
6
7  plt.hist(x, 50, density=True, color="g", alpha=0.75)
8
9  plt.xlabel('Smarts')
10 plt.ylabel('Probability')
11 plt.title('Histogram of IQ')
12 plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
13 plt.xlim(40, 160)
14 plt.ylim(0, 0.03)
15 plt.grid(True)
16 plt.show()
```



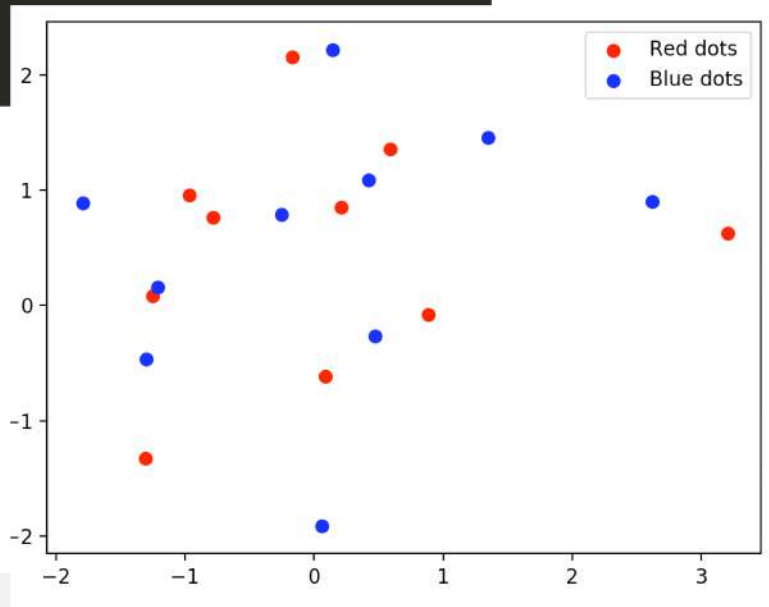
Scatter plot

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.random.randint(50,size=20)
5  y = np.random.randint(50,size=20)
6  plt.scatter(x,y)
7  plt.title("Random scatter plot")
8  plt.show()
```



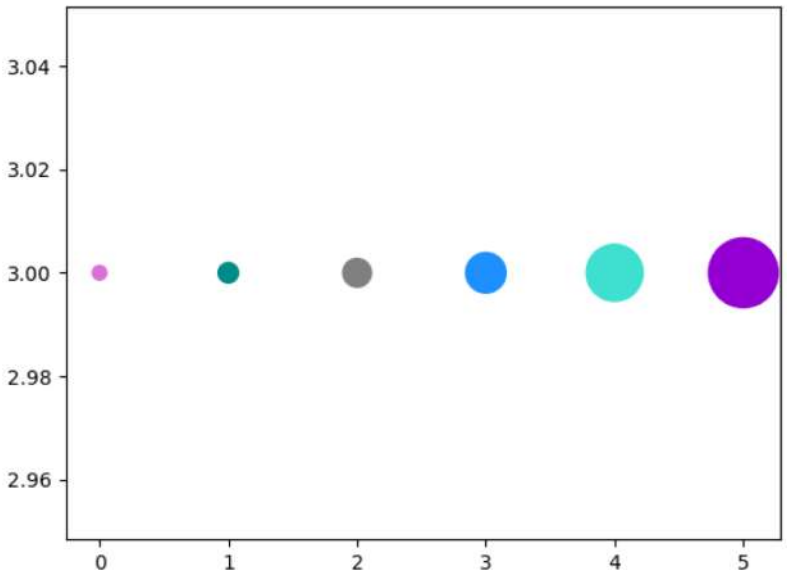
Scatter plot

```
1  import matplotlib.pyplot  as plt
2  import numpy as np
3
4  x = np.random.randn(10)
5  y = np.random.randn(10)
6  plt.scatter(x,y,c="r",label="Red dots")
7  x = np.random.randn(10)
8  y = np.random.randn(10)
9  plt.scatter(x,y,c="b",label="Blue dots")
10 plt.legend()
11 plt.show()
```



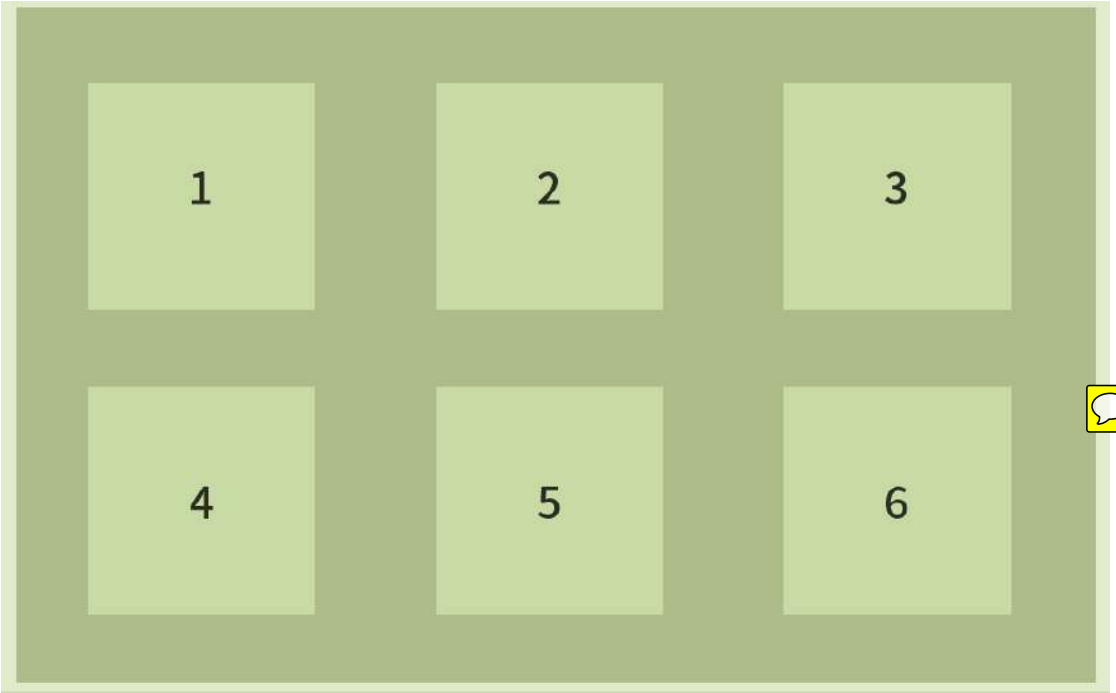
Scatter plot

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 x  = np.arange(0,6)
4 y = [3,3,3,3,3,3]
5 size_map = [50,100,200,400,800,1200]
6 color_map = [ 'orchid', 'darkcyan', 'grey', 'dodgerblue', 'turquoise','darkviolet']
7 plt.scatter(x,y,sizes = size_map, c=color_map)
8 plt.show()
```



Advanced technique: Subplots

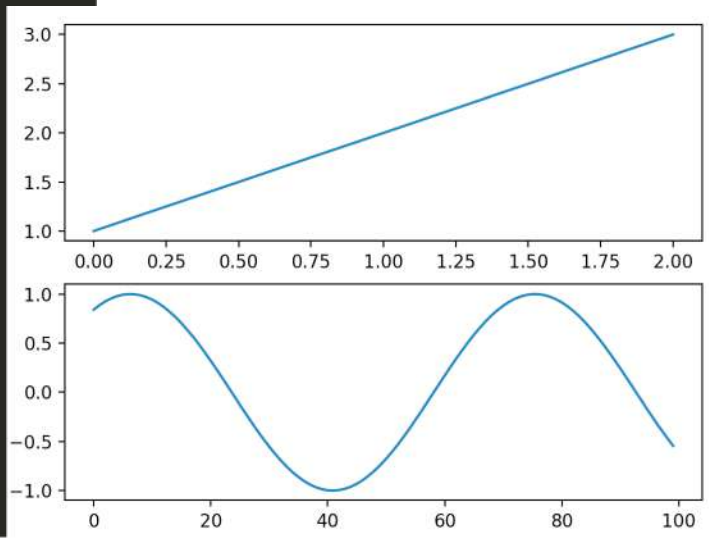
- If you want to show multiple plots at the same time
  - Use plt.subplots(row,column,number)
- The order follows from left to right, up to down



Advanced technique: Subplots

- If you want to show multiple plots at the same time
  - Use plt.subplots(row,column,number)
- The order follows from left to right, up to down

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  #first plot
5  plt.subplot(211)
6  plt.plot([1,2,3])
7
8  #second plot
9  plt.subplot(212)
10 x = np.sin(np.linspace(1,10,100))
11 plt.plot(x)
12 plt.show()
```



Save your plots

圖片大小

```
plt.savefig("piechart", dpi = 500)
```

檔案名稱